

**Yenepoya Institute of Arts, Science,
Commerce and Management**

Project – Low Level Design

on

**Bigdata Analytics on Smart City Demand
Analysis**

Student Name: Fathimath Shakiya A S

**Industry
Mentor:**

Sumit Kumar Shukla

Guided by: Prashantha J

1. Introduction

This document describes the detailed implementation of the Smart City Demand Analytics system using Python, Pandas, Seaborn, and SQLite. The system processes urban data to analyze demand patterns such as traffic, electricity, and water usage.

2. Scope of the Document

This document covers:

- Step-by-step data processing
- Data transformation and analysis
- Visualization techniques
- Database storage and querying

3. System Overview

The system works on a CSV dataset and performs:

- Data loading using Pandas
- Data cleaning and preprocessing
- Feature creation
- Data analysis and visualization
- Storage using SQLite

4. Implementation Design (Step-Based)

4.1 Data Loading

- Dataset is loaded using Pandas:

```
df = pd.read_csv("dataset.csv")
```

4.2 Data Cleaning

- Missing values are handled using:

```
num_cols = df.select_dtypes(include=['float64', 'int64']).columns
```

```
df[num_cols] = df[num_cols].fillna(0)
```

```
cat_cols = df.select_dtypes(include=['object', 'string']).columns
```

```
df[cat_cols] = df[cat_cols].fillna('Unknown')
```

4.3 Data Analysis

- Aggregation using GroupBy:

```
df.groupby('hour')['traffic_volume'].mean()
```

- Correlation analysis:

```
df.corr()
```

4.4 Data Visualization

Visualization is performed using Seaborn:

- Line plots (traffic trends)
- Bar charts (comparisons)
- Heatmaps (correlation and traffic patterns)

Example:

```
sns.lineplot(x='hour', y='traffic_volume', data=df)
```

4.5 Database Storage

- Data is stored in SQLite database:

```
import sqlite3
```

```
conn = sqlite3.connect('smart_city.db')
```

```
df.to_sql('demand_data', conn, if_exists='replace', index=False)
```

4.6 SQL Query Execution

- Example query:

```
SELECT hour, AVG(traffic_volume) as avg_traffic  
FROM demand_data  
GROUP BY hour;
```

5. Process Flow

```
CSV Dataset  
↓  
Data Loading (Pandas)  
↓  
Data Cleaning  
↓  
Analysis (GroupBy, Correlation)  
↓  
Visualization (Seaborn)  
↓  
SQLite Storage  
↓  
SQL Queries
```

6. Data Design

Key fields used:

- hour, day_of_week
- traffic_volume
- weather_main
- temp, temp_celsius
- demand levels

7. Database Design

Table: demand_data

Column	Type
hour	INTEGER
day_of_week	INTEGER
traffic_volume	INTEGER
temp_celsius	FLOAT
weather_main	TEXT

8. Data Access Mechanism

- Data processed using Pandas DataFrame
- Data stored and accessed using SQLite queries

9. Interfaces

- Jupyter Notebook
- Graphical outputs (charts and heatmaps)

10. Error Handling

- Missing values handled using fillna()
- Basic data validation during processing

11. Performance Considerations

- Efficient aggregation using Pandas
- Fast query execution using SQLite

12. Security Considerations

- No sensitive data used
- All processing is local

13. Conclusion

The system provides a simple and efficient implementation of smart city demand analytics using Python tools, following a structured ETL and analysis workflow.